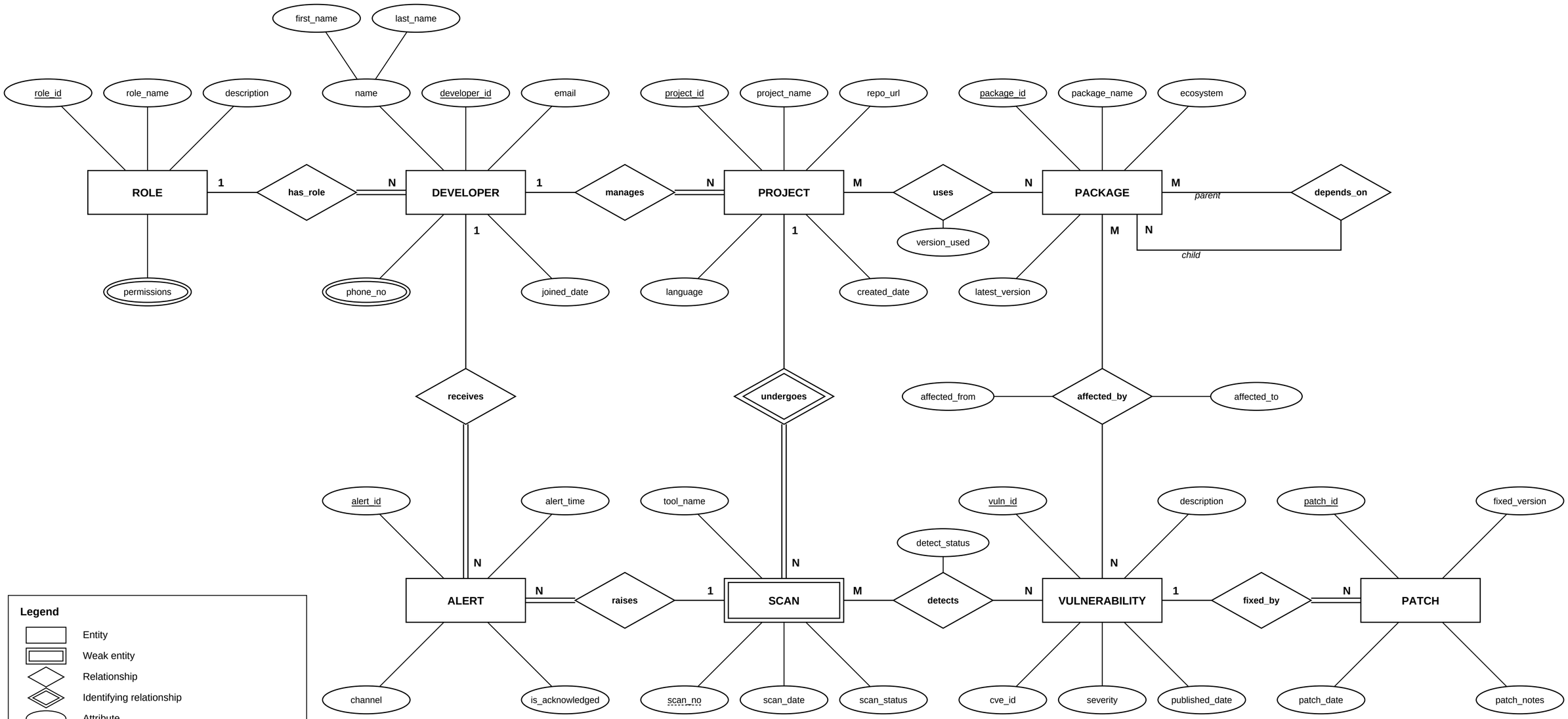
ER Diagram: Software Dependency Vulnerability Tracker

DBMS Mini Project | Chen notation | 8 entities, 10 relationships



Legend

- Entity
- Weak entity
- Relationship
- Identifying relationship
- Attribute
- Key attribute (underlined)
- Partial key (dashed underline)
- Multivalued attribute
- Total participation (double line)

Cardinality: 1, N, M written beside each entity

Entities and attributes

Entity	Attribute	Type	Meaning
ROLE	role_id	Key	Unique id of the role
	role_name	Simple	admin / developer / viewer
	description	Simple	What the role is for
	permissions	Multivalued	Several permissions per role (read, write, run_scan ...)
DEVELOPER	developer_id	Key	Unique id of the developer
	name	Composite	Made of first_name and last_name
	email	Simple	Login / contact email
	phone_no	Multivalued	A developer can have more than one phone number
	joined_date	Simple	Date the developer joined
PROJECT	project_id	Key	Unique id of the project
	project_name	Simple	Name of the software project
	repo_url	Simple	Git repository link
	language	Simple	Main programming language
	created_date	Simple	Date the project was registered
PACKAGE	package_id	Key	Unique id of the package
	package_name	Simple	e.g. express, mongodb, react
	ecosystem	Simple	npm / PyPI / Maven
	latest_version	Simple	Newest released version
SCAN (weak)	scan_no	Partial key	Number of the scan within one project (project_id + scan_no identifies it)
	scan_date	Simple	When the scan ran
	scan_status	Simple	completed / failed / running
	tool_name	Simple	Scanner used
VULNERABILITY	vuln_id	Key	Unique id of the vulnerability
	cve_id	Simple	Public CVE code, e.g. CVE-2021-44228 (unique)
	severity	Simple	LOW / MEDIUM / HIGH / CRITICAL
	published_date	Simple	Date the CVE was published
	description	Simple	Short text about the weakness
PATCH	patch_id	Key	Unique id of the patch
	fixed_version	Simple	Package version that contains the fix
	patch_date	Simple	Date the fix was released
	patch_notes	Simple	Release notes of the fix
ALERT	alert_id	Key	Unique id of the alert
	alert_time	Simple	When the alert was raised
	channel	Simple	email / dashboard / slack
	is_acknowledged	Simple	Whether the developer has seen it

Relationships

Relationship	Entities	Cardinality	Total participation	Relationship attribute
has_role	ROLE - DEVELOPER	1 : N	Developer total	-
manages	DEVELOPER - PROJECT	1 : N	Project total	-
uses	PROJECT - PACKAGE	M : N	Partial	version_used
depends_on	PACKAGE - PACKAGE (recursive)	M : N	Partial	- (roles: parent, child)
affected_by	PACKAGE - VULNERABILITY	M : N	Partial	affected_from, affected_to
undergoes (identifying)	PROJECT - SCAN	1 : N	Scan total	-
detects	SCAN - VULNERABILITY	M : N	Partial	detect_status
fixed_by	VULNERABILITY - PATCH	1 : N	Patch total	-
raises	SCAN - ALERT	1 : N	Alert total	-
receives	DEVELOPER - ALERT	1 : N	Alert total	-

Notes for the viva

• SCAN is a weak entity: it has no full key of its own. Its identity is project_id (from PROJECT) + scan_no, and the identifying relationship is undergoes.

• Every M : N relationship (uses, depends_on, affected_by, detects) becomes its own table when converting to the relational schema, with a composite primary key made of the two foreign keys. Relationship attributes (version_used, affected_from, affected_to, detect_status) become columns of that table.

• 1 : N relationships become a foreign key on the N side (e.g. manager_id in PROJECT, role_id in DEVELOPER, vuln_id in PATCH, project_id in SCAN, scan_id and developer_id in ALERT).

• name is composite (first_name, last_name), so it maps to two columns. permissions and phone_no are multivalued, so each maps to its own table (role_permission, developer_phone).

• depends_on is a recursive relationship on PACKAGE. It is what the recursive CTE walks to build the full transitive dependency tree.

• The role entity supports role-based access control (admin, developer, viewer) required by the rubric.